# Memory Management in C and Managed Languages

Memory management determines how your program obtains, uses, and releases RAM during execution.

We will compare low-level control in C with managed runtimes on the JVM and CPython.

MANUAL CONTROL — AUTOMATIC MANAGEMENT

## C

EXPLICIT CONTROL

Done manually with explicit allocation and deallocation.

Offers precise, low-level control over hardware resources.

## Java

GARBAGE COLLECTED

Largely automatic memory management via garbage collection.

Managed runtime operating on the JVM.

## Python

REFERENCE COUNTING

Largely automatic memory management via garbage collection.

Managed runtime operating on CPython.

# Agenda

01 Process memory layout: code, data, stack, heap.

02 Manual memory management in C: stack vs heap, malloc/free, common bugs.

03 Managed memory in Java: heap generations, garbage collectors, tuning.

04 Managed memory in Python: reference counting plus cyclic GC.

05 Best practices and crossâ€‘language comparisons.

# Process Memory Layout

A typical C process divides memory into regions: **code (text)**, **static/global data**, **heap**, and **stack**.

The **stack** holds call frames (parameters, return addresses, local variables); it grows and shrinks automatically as functions call and return.

The **heap** holds dynamically allocated objects whose lifetime is controlled by allocation and deallocation calls.

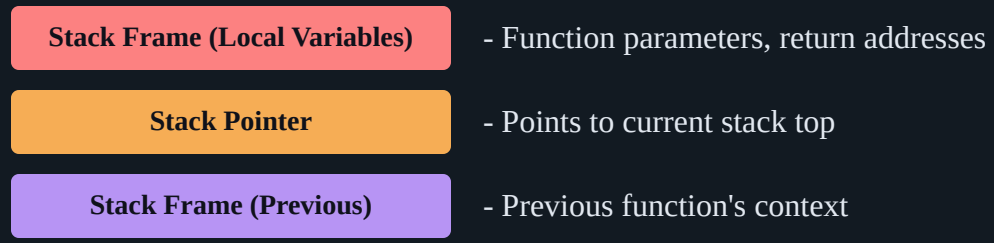

- Program instructions
- Global variables
- Dynamic allocation
- Local variables

# Stack in C

Local variables and function parameters are stored in **stack frames**, managed automatically by the compiler.

A variable like `int i;` declared inside a function lives on the stack and is destroyed when the function returns.

Taking the address of such a variable (e.g., `&i`) gives a pointer into the stack frame that becomes **invalid after the function exits**.



# Heap in C

The **heap region** is reserved for objects created at runtime via `malloc`, `calloc`, `realloc`, and destroyed with `free`.

Heap allocations **outlive** the function that created them and can be shared across functions via pointers.

Failing to **free** heap memory leads to **leaks**; freeing the wrong address **corrupts** the allocatorâ€™s internal structures.

```
// Heap allocation example
int *arr = malloc(10 * sizeof(int));
free(arr); // deallocate
```

# Static and Global Storage

**Global variables** and objects with `static storage duration` live in a separate data/static region and exist for the entire program run.

They are initialized at **program startup** and never explicitly freed by user code. Static variables within functions retain their values between function calls.

**Overuse of global/static data** can increase memory footprint and reduce modularity. Global variables can be accessed from any part of the program, which may lead to naming conflicts.

# Manual Allocation Basics in C

```c
// Dynamic allocation example
int *arr = malloc(10 * sizeof(int));  // allocate
if (!arr) { /* handle out-of-memory */ }
/* use arr[0..9] */
free(arr);                            // deallocate
```

**malloc** returns a pointer to a block of raw bytes; the programmer decides how to interpret and initialize it.

It is the programmer's responsibility to ensure every successful allocation is eventually freed exactly once.

**Key Points:**

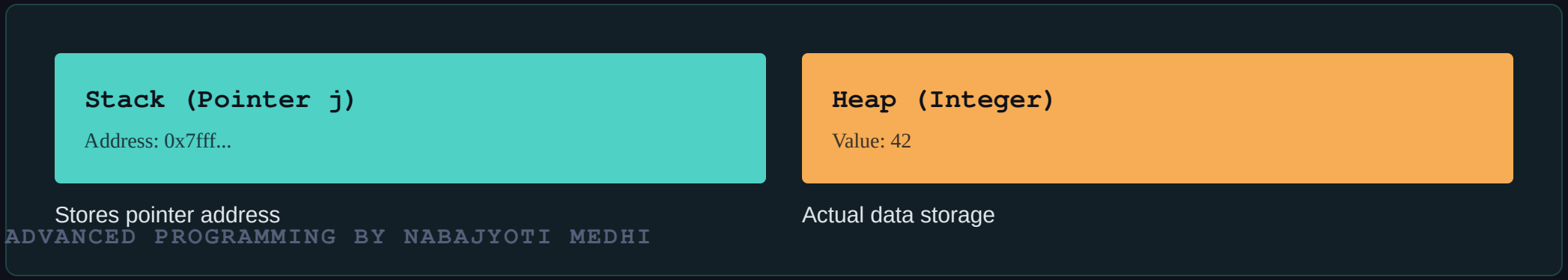
- Check for NULL after malloc
- Free memory when done
- Don't access after free

# Pointers on Stack, Data on Heap

When you write `int *j = malloc(sizeof(int));`, the pointer variable **j** lives on the stack, but the integer it points to is on the heap.

The stack slot for **j** stores the address returned by `malloc`, not the integer value itself.

Losing all pointers to a heap block (e.g., reassigning **j** without freeing) causes a **memory leak** in C.



# Simple C Example: Dynamic Array

```c
int n = 5;
int *a = malloc(n * sizeof(int));
for (int i = 0; i < n; i++) a[i] = i * i;
/* use a */
free(a);
```

The array elements live in one **contiguous heap block**; their lifetime is independent of the stack frame's lifetime.

**Forgetting free(a)** or calling free twice on a leads to undefined behavior and possible crashes.

**Key Points:**

- Allocate with malloc
- Access elements via pointer
- Free when done

# Common Heap Bugs in C

**Memory leak:** allocated memory is never freed, causing the process's heap usage to grow over time.

**Dangling pointer:** pointer used after the underlying heap block has been freed (use-after-free).

**Invalid free:** passing an address that never came from malloc/calloc/realloc to free corrupts allocator metadata.

**Double free:** freeing the same memory block twice, leading to undefined behavior and potential crashes.

**Buffer overflow:** writing past the end of an allocated block, corrupting adjacent memory.

Critical  Warning

# Tools for C Memory Debugging

## Valgrind

Dynamic analysis tools like Valgrind can detect leaks, use-after-free, and invalid frees by instrumenting heap operations.

## AddressSanitizer (ASan)

AddressSanitizer and related sanitizers in modern compilers catch out-of-bounds accesses and memory errors at runtime.

## Essential Tools

These tools are essential when working with manual memory management in large C codebases.

# Stack vs Heap Trade-offs

### Stack

Automatic, fast allocation

Limited in size

Best for small, short-lived data

### Heap

Flexible, larger structures

Slower, error-prone if misused

Requires manual management

**Key Insight:** Understanding when to choose stack vs heap affects **performance**, **safety**, and **clarity** of C code. Stack allocation is automatic and fast but limited; heap allocation is flexible but requires careful management.

# Structs and Dynamic Allocation in C

```c
typedef struct {
    char *name;
    int  age;
} Person;

Person *p = malloc(sizeof(Person));
p->age = 30;
/* allocate and copy name separately */
free(p);
```

**Nested heap allocations** (e.g., name) require freeing each allocated subâ€‘object to avoid leaks.

**Key Concepts:**

- Heap-allocated structs
- Nested allocations
- Ownership design
- Memory cleanup

Designing clear ownership (who allocates, who frees) is critical in manual memory management.

# Buffers and Overflows in C

**Buffer overflow** occurs when data exceeds the allocated buffer size, overwriting adjacent memory locations.

**Classic vulnerability:** Writing past the end of `char buf[16];` when reading input without length checks.

Buffer overflows are a major source of **security vulnerabilities** in low-level languages like C.

```
// Vulnerable code example
void vulnerable_function() {
    char buffer[16];
    // Danger: reads input without checking length
    gets(buffer);  // NEVER use gets()!
}
```

**Security Impact:** Buffer overflows can lead to arbitrary code execution, data corruption, and system crashes. Always use `fgets()` or `scanf()` with length limits.

# RAII-Style Cleanup Pattern in C

```c
int func(void) {
    int *buf = malloc(100 * sizeof(int));
    if (!buf) return -1;
    /* ... work ... */
cleanup:
    free(buf);
    return 0;
}
```

**C does not have RAII** in the language, but you can emulate it with structured cleanup using a single exit path and labeled cleanup.

This pattern reduces the chance of missing `free` on error paths and helps manage resource lifetime centrally.

**Key Benefits:**

- Single exit point for cleanup
- Avoids missing free on errors
- Centralized resource management

# Custom Allocators and Arenas

For **performance**, C programs sometimes build arena or pool allocators on top of malloc to group related allocations.

An **arena** lets you allocate many objects and then free them all at once by resetting or destroying the arena.

This trades fine-grained control for faster bulk deallocation and simpler lifetime rules.

Common use cases: **game engines**, **web servers**, and **data processing pipelines** where many objects share similar lifetimes.

```
// Arena allocator example
typedef struct {
    char *buffer;
    size_t used;
    size_t capacity;
} Arena;

void *arena_alloc(Arena *a, size_t size) {
    if (a->used + size > a->capacity) return NULL;
    void *ptr = a->buffer + a->used;
    a->used += size;
    return ptr;
}
```

# What Is a Managed Language?

A **managed language** runs atop a runtime that automatically allocates and reclaims heap memory, hiding explicit `free` from the programmer.

**Java** and **Python** provide automatic garbage collection so that unreachable objects are reclaimed without manual intervention.

The runtime tracks object graphs from **roots** (threads, stacks, globals) instead of relying on programmer-written free calls.

### Memory Safety

Automatic memory management prevents common bugs like memory leaks and dangling pointers.

### Performance

Optimized GC algorithms minimize pause times for responsive applications.

### Productivity

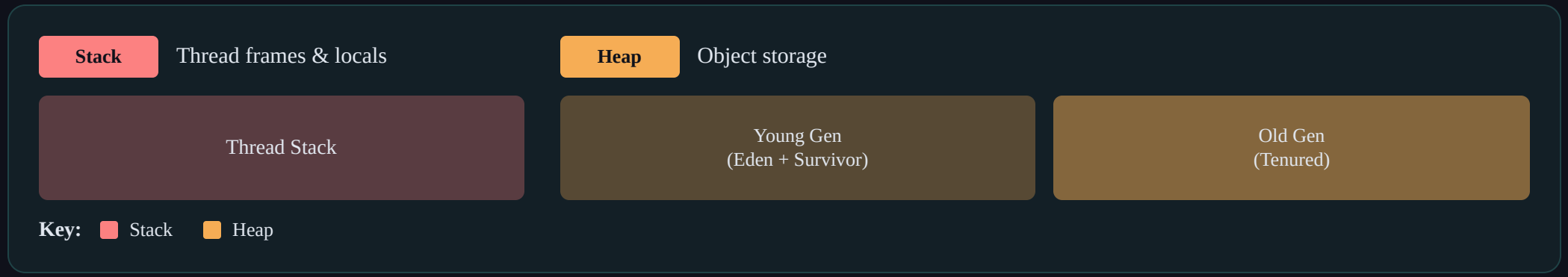
Focus on business logic instead of manual memory management.

# Java Memory Model Overview

Java code runs on a **JVM** which manages a **heap** for objects and a **stack** for each thread's frames and local variables.

Objects are always allocated on the heap via **new**, while primitives in local variables typically live on the stack frame.

The heap is partitioned into **generations** (young, old/tenured) to optimize collection of short-lived vs long-lived objects.



# Java Object Lifecycle

```java
// Java object lifecycle example
Person p = new Person("Alice");
p.doSomething();
p = null;    // remove reference
// later, GC may reclaim the Person object
```

When an object becomes **unreachable** from any GC root, it is considered garbage and becomes eligible for collection.

The programmer does not call `free`; the GC eventually reclaims dead objects and may compact the heap.

**Key Points:**

- Objects allocated with `new`
- GC reclaims unreachable objects
- Heap may be compacted

# Generational Garbage Collection in Java

Generational GC divides the heap into a **young generation** (Eden + survivor spaces) and an **old/tenured generation**.

New objects are allocated in Eden; frequent minor collections clean up shortâ€‘lived objects there.

Objects that survive several collections are promoted to the old generation, which is collected less frequently in major or full GCs.

# Modern Java Collectors (High-Level)

Newer JVMs include collectors like **G1**, **ZGC**, and **Shenandoah** that aim for lowâ€‘pause, concurrent collection.

**Generational ZGC**, for example, uses a twoâ€‘generation heap and concurrent marking with colored pointers and barriers to minimize pauses.

Choosing and tuning collectors is a key part of JVM performance engineering for large systems.

| | |
|---|---|
| **G1 GC** | - Region-based, incremental collection |
| **ZGC** | - Ultra-low latency, scalable |
| **Shenandoah** | - Concurrent evacuation, low pause |

# Heap Size and Tuning in Java

JVM options like -Xms and -Xmx control the initial and maximum heap size available for object allocation.

**Too small a heap can cause frequent GC and high overhead; too large a heap can increase full GC pause times.**

Profilers and GC logs help you choose appropriate heap sizes and GC algorithms for your workload.

```
// Initial heap size
-Xms512m

// Maximum heap size
-Xmx4g

// Young generation size
-Xmn1g

// GC algorithm
-XX:+UseG1GC
```

- Monitor with: -XX:+PrintGCDetails and -Xlog:gc*

# Memory Leaks in Managed Java

Even with GC, Java programs can leak memory by unintentionally retaining references to unused objects (e.g., in static maps or longâ€‘lived caches).

Event listener lists, threadâ€‘local storage, and collections that grow without bounds are common leak sources.

Tools like VisualVM, Java Flight Recorder, and heap dump analyzers help detect such retained objects.

# Java Example: Auto Memory vs Explicit Close

```java
// Example combining GC with manual resource release
try (FileInputStream in = new FileInputStream("data.bin")) {
    // use stream
}   // auto close via try-with-resources
```

The FileInputStream object is **GC-managed**, but its underlying OS handle must be **explicitly closed** or released by finalization/close methods.

Memory GC and external resources management are **related but distinct concerns** in managed languages.

**Key Points:**

- try-with-resources ensures auto-close
- GC manages object memory
- External resources need explicit cleanup

# Python Memory Model Overview

CPython uses **automatic memory management** built around reference counting and a generational garbage collector.

Variables are **references** to objects located in the heap; assignments bind names to existing objects rather than copying raw bytes.

The interpreter maintains metadata for each object, including a **reference count** and type information.

Python's memory model is **managed** - you don't explicitly allocate or free memory like in C.

| | |
|---|---|
| **Reference Count** | - Tracks how many references point to an object |
| **Generational GC** | - Handles cyclic references |
| **Heap Objects** | - All Python objects live on the heap |

# Reference Counting in Python

```python
import sys
x = [1, 2, 3]
print(sys.getrefcount(x))  # 2 (one from x, one from getrefcount argument)
y = x
print(sys.getrefcount(x))  # 3 (increased by y)
del y
print(sys.getrefcount(x))  # 2 (decreased)
```

**Reference counting** tracks how many references point to each object; when the count drops to zero, the object is immediately deallocated.

Each **sys.getrefcount()** call creates a temporary reference, so the count is always at least 2 for existing variables.

**Key Points:**

- Immediate deallocation on refcount=0
- Deterministic destruction
- Cannot handle cycles alone

# Cyclic Garbage Collection in Python

Circular references (e.g., two objects referencing each other) never reach zero reference count, so they are handled by a **separate cyclic GC**.

Python groups objects into **generations (0, 1, 2)** and periodically scans younger generations for unreachable cycles.

The **gc module** exposes controls to inspect and trigger collection manually for debugging.

| | | |
|---|---|---|
| `Gen 0 (Young)` | New objects, collected most frequently | `Threshold: 700 allocations` |
| `Gen 1 (Medium)` | Survived 1+ collections | `Threshold: 10 collections` |
| `Gen 2 (Old)` | Survived 2+ collections, collected least frequently | `Threshold: 10 collections` |

**âœ“ Automatic Detection**

GC automatically finds unreachable cycles

**âœ“ Manual Control**

gc.collect() for debugging

# Python Example: Cycles

```
class Node:
    def __init__(self):
        self.ref = None
a = Node()
b = Node()
a.ref = b
b.ref = a  # cycle
import gc
del a, b
gc.collect()  # breaks the cycle
```

**Cyclic references** (e.g., two objects referencing each other) never reach zero reference count, so they are handled by a separate cyclic GC.

Without the cyclic GC, these objects would remain leaked due to positive reference counts.

**Key Points:**

- Cyclic GC handles circular refs
- Generational collection
- Manual gc.collect() call

# Python Memory Pitfalls

**Long-lived global variables and caches** can keep large objects alive even when no longer needed, effectively leaking memory.

**Reference cycles** involving objects with custom `__del__` methods can complicate collection and may be left uncollected.

**Tools for debugging:** **tracemalloc**, **objgraph**, and **gc.get_objects()** help inspect memory usage.

> **Common Pitfall:** Objects with `__del__` methods in cycles may not be collected properly. Consider using `weakref` for circular references.

**tracemalloc**
Track memory allocations

**objgraph**
Visualize object graphs

**gc module**
Control GC behavior

# C vs Java vs Python: Allocation Model

## C

(MANUAL)

You choose stack vs heap explicitly; heap allocations use `malloc` / `free` and are completely under programmer control.

## Java

(MANAGED)

Almost all objects are allocated on the managed heap with `new` and reclaimed automatically by the garbage collector.

## Python

(MANAGED)

Objects live on a managed heap with automatic reclamation via reference counting and cyclic GC.

# Deterministic vs Non-Deterministic Destruction

**In C**, you decide exactly when to free a block; destruction is deterministic but error-prone. The programmer has full control over memory deallocation timing.

**In Java and Python**, the exact time when an unreachable object is reclaimed is generally unspecified and depends on GC heuristics and runtime conditions.

**Python's reference counting** can reclaim some objects immediately, but cycles and GC scheduling still introduce non-determinism overall.

**Key difference**: Manual control vs. automatic management - C gives predictability at the cost of safety, while managed languages prioritize safety with less control.

| C (Deterministic) | Java/Python (Non-Deterministic) |
| --- | --- |
| Explicit free() calls | GC reclaims objects |
| Immediate deallocation | Timing depends on GC |
| Predictable timing | Unpredictable delays |

# Managing Non-Memory Resources

## C (RAII-style)

Use structs and functions to tie resource acquisition and release to scope or explicit free functions.

```c
struct File {
  FILE* fp;
  char* path;
};

void file_close(File* f) {
  fclose(f->fp);
  free(f->path);
}
```

## Java (try-with-resources)

Ensures close() is called even if exceptions occur, decoupling resource lifetime from GC.

```java
try (FileInputStream in =
    new
FileInputStream("data.bin"))
{
  // use stream
} // auto close
```

## Python (with statement)

Provides structured release of files, locks, and network connections.

```python
with open('file.txt') as
f:
  content = f.read()
# f closed automatically
```

# Memory and Exceptions

In **C**, manual cleanup must be carefully structured so that all `malloc`ed blocks are freed on every error path, often using `goto cleanup` or similar patterns.

**Java** and **Python** unwind the stack automatically on exceptions, but heap objects referenced from surviving frames or globals remain live.

Using `finally` / `try-with-resources` in Java and `with` / `try/finally` in Python prevents resource leaks when errors occur.

### C Error Handling

```c
int func(void) {
  int *buf = malloc(100 * sizeof(int));
  if (!buf) return -1;
  if (error) { free(buf); return -1; }
  free(buf);
}
```

### Java Error Handling

```java
try (FileInputStream in = new
FileInputStream("file")) {
  // use stream
} catch (IOException e) {
  handleError(e);
}
```

# Multithreading and Memory Safety

### C

C's manual memory management combined with threads can cause data races, double frees, and inconsistent views of shared heap data if not synchronized.

### Java

Java's memory model, synchronized blocks, and thread-safe constructs help reason about visibility of object updates across threads.

### Python

Python's GIL simplifies some CPython data structure safety, but shared mutable objects across threads still require careful design.

# Interfacing C with Java (JNI)

JNI allows Java code to call **native C functions** and share data across managed and unmanaged heaps.

Java objects passed into native code remain under **GC control**, while C allocations must still be manually freed.

Careful design avoids **leaking native memory** or using Java objects after the JVM considers them unreachable.

```
// JNI function signature
JNIEXPORT void JNICALL Java_MyClass_nativeMethod(
  JNIEnv* env, jobject obj, jint param) {
  // Access Java object fields
  jclass cls = (*env)->GetObjectClass(env, obj);
  jfieldID fid = (*env)->GetFieldID(env, cls, "field", "I");
  jint value = (*env)->GetIntField(env, obj, fid);
  // Must free local references
  (*env)->DeleteLocalRef(env, cls);
}
```

### Key JNI Concepts

**JNIEnv:** Interface pointer for JNI functions

**jobject:** Reference to Java object

**jfieldID:** Field identifier for access

**Local Refs:** Must be freed to prevent leaks

### âš ï¸ Common Pitfalls

Forgetting to delete local references explicitly within the JNI code.

Accessing Java objects or array elements after the GC has potentially moved them.

Memory leaks originating entirely in native code operations.

# Interfacing C with Python (C API)

Python's C API lets you create extension modules that allocate C structures and expose them as Python objects.

Extension code must obey Python's reference counting protocol (increment/decrement reference counts correctly) to avoid leaks or crashes.

Mixing C heap objects with Python's managed heap requires a disciplined ownership model.

```
// Example: Creating a Python extension
static PyObject* my_func(PyObject* self, PyObject* args) { int
value; if (!PyArg_ParseTuple(args, "i", &value)) return NULL;
return PyLong_FromLong(value * 2); }
```

### Key Concepts

- Reference counting (Py_INCREF/Py_DECREF)
- Memory management rules
- Error handling

**Important:** Always check return values and handle errors properly to prevent memory leaks.

# Case Study: Dynamic Array in C

```c
typedef struct {
    int *data;
    size_t size, capacity;
} IntVec;
// Initialize dynamic array
IntVec* intvec_new(size_t capacity) {
    IntVec* v = malloc(sizeof(IntVec));
    if (!v) return NULL;
    v->data = malloc(capacity * sizeof(int));
    if (!v->data) { free(v);   return NULL; }
    v->size = 0;   v->capacity = capacity;
    return v;
}
// Resize when capacity is exceeded
int intvec_push(IntVec* v, int val) {
    if (v->size >= v->capacity) {
        size_t new_cap = v->capacity * 2;
        int* new_data = realloc(v->data, new_cap * sizeof(int));
        if (!new_data) return -1;
        v->data = new_data;    v->capacity = new_cap;
    }   v->data[v->size++] = val;
    return 0;
```

### Key Implementation Details

â€¢ **Data pointer**: Points to heap-allocated array
â€¢ **Size**: Current number of elements
â€¢ **Capacity**: Total allocated space

### âš ï¸ Common Bugs

â€¢ Forgetting to free data in destructor
â€¢ Double free on resize failure
â€¢ Memory leak on init failure

# Case Study: Java ArrayList

```java
public class ArrayList<E> implements List<E> {
    private Object[] elementData;  // backing array
    private int size;
    private static final int DEFAULT_CAPACITY = 10;

    public ArrayList() {
        this.elementData = new Object[DEFAULT_CAPACITY];
    }

    public boolean add(E e) {
        ensureCapacityInternal(size + 1);
        elementData[size++] = e;
        return true;
    }

    private void ensureCapacityInternal(int minCapacity) {
        if (minCapacity - elementData.length > 0) {
            grow(minCapacity);
        }
    }
    private void grow(int minCapacity) {
        int oldCapacity = elementData.length;
        int newCapacity = oldCapacity + (oldCapacity >> 1);
        if (newCapacity - minCapacity < 0)
            newCapacity = minCapacity;
        elementData = Arrays.copyOf(elementData, newCapacity);
    }
}
```

### Key Implementation Details

â€¢ **Object[]**: Generic array storing elements
â€¢ **size**: Current number of elements
â€¢ **capacity**: Total allocated space
â€¢ **grow()**: 1.5x capacity increase

### âš ï¸ Memory Considerations

â€¢ GC tracks all references
â€¢ Clear references to allow GC
â€¢ TrimToSize() to reduce memory

# Case Study: Python List

```python
# Python list implementation
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None

# Create a list with references
my_list = [1, 2, 3, 4, 5]

# List operations
my_list.append(6)           # Add element
my_list.pop()               # Remove last
my_list.insert(0, 0)        # Insert at index

# Reference counting
import sys
print(sys.getrefcount(my_list))  # Shows reference count
```

### Python List Characteristics

â€¢ **Dynamic array**: Contiguous memory for references
â€¢ **Reference counting**: Automatic memory management
â€¢ **Growth factor**: Over-allocation for amortized O(1)

### âš ï¸ Memory Considerations

â€¢ List resizing may cause temporary memory spikes
â€¢ Large lists can trigger GC cycles

# Performance: Manual vs GC

**Manual memory management** can be more predictable and efficient when done correctly, especially in tight loops and systems programming.

**GC adds overhead** (marking, sweeping, compaction) and can introduce pause times, but it reduces programmer burden and certain classes of bugs.

**Modern collectors** (G1, ZGC, etc.) aim to minimize pause times with concurrent and incremental techniques.

| Manual Management | Garbage Collection | Modern GC |
| --- | --- | --- |
| Predictable performance | Automatic memory handling | Concurrent marking |
| No GC pauses | May cause pauses | Low pause times |
| Requires expertise | Lower bug risk | Scalable |

# Fragmentation and Compaction

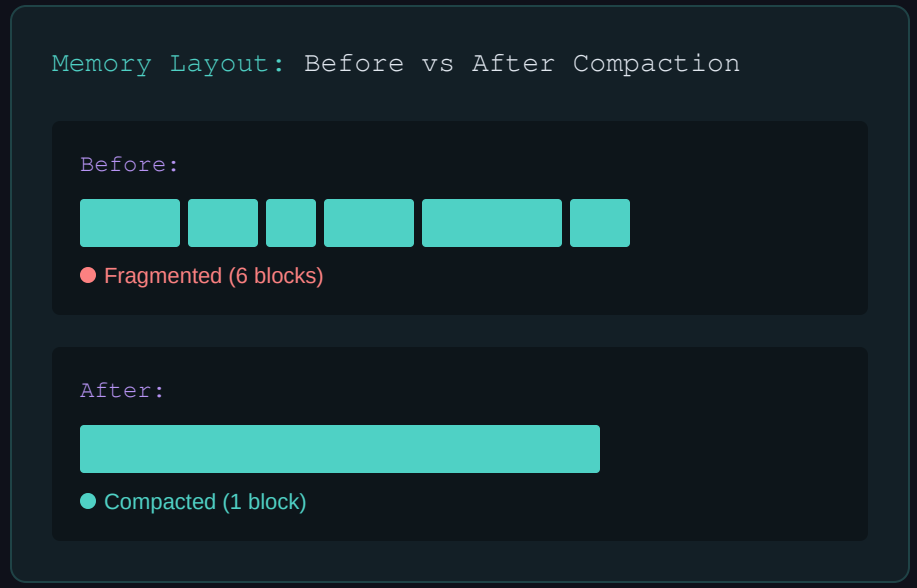
**Heap Fragmentation:** In C, repeated allocations and frees can fragment the heap, leaving many small free blocks and hurting locality.

**Memory Compaction:** Some GC algorithms compact live objects to reduce fragmentation and improve cache behavior.

**Object Movement:** Compacting moves objects, so managed languages use references that can be updated transparently by the runtime.

**Cache Performance:** Contiguous memory layout improves CPU cache utilization and overall performance.

> **Key Insight:** Manual memory management in C can lead to fragmentation, while GC-based systems can compact memory to maintain efficiency.



# Out-of-Memory Handling

**C:** `malloc` may return `NULL` to signal failure; robust C code must check and handle this explicitly.

**Java:** Allocation failures typically manifest as `OutOfMemoryError` when the heap cannot grow or GC cannot free enough space.

**Python:** Raises `MemoryError` when it cannot satisfy an allocation request on its managed heap.

**C (malloc NULL check)**

```c
int *arr = malloc(100 * sizeof(int));
if (arr == NULL) {
  fprintf(stderr, "Out of memory\n");
  return -1;
}
```

**Java (OutOfMemoryError)**

```java
try {
  byte[] large = new byte[1000000000];
} catch (OutOfMemoryError e) {
  System.err.println("OOM: " + e.getMessage());
}
```

**Best Practice: Always check allocation results**

**C:** Check malloc return value before dereferencing **Java:** Catch OutOfMemoryError, free resources **Python:** Handle MemoryError in try-except

# Security Implications

**C's manual memory and raw pointers** enable powerful low-level code but also enable buffer overflows, use-after-free, and arbitrary memory writes when misused.

**Managed languages** typically prevent direct out-of-bounds writes and dangling references, reducing whole classes of memory-safety vulnerabilities.

**Logic errors** (e.g., deserialization bugs, injection) still occur regardless of memory management strategy.

| LANGUAGE | MEMORY SAFETY | COMMON VULNERABILITIES |
|---|---|---|
| C | Manual (High Risk) | Buffer overflow, use-after-free |
| Java | Managed (Safe) | Logic errors, injection |
| Python | Managed (Safe) | Logic errors, injection |

# GC Pauses and Latency

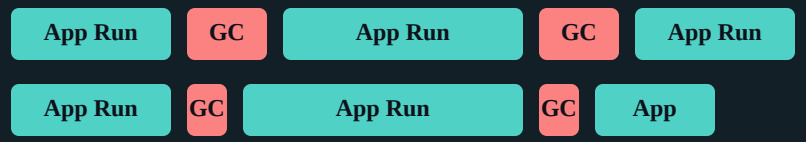
**Stop-the-world GC pauses** in Java can impact latency-sensitive services if not tuned correctly. These pauses occur when the GC must halt all application threads to perform collection.

**Modern collectors** try to perform marking and evacuation concurrently to keep pauses short, at the cost of more complex algorithms.

**Python's cyclic GC** also runs periodically; while usually lightweight, high-allocation workloads may need tuning.

**Key consideration:** GC pause times are often more critical than total GC time for interactive applications.

**GC Pause Timeline Example:**




Green = Application running, Red = GC pause (shorter is better)

# Memory Profiling Tools

**C** Valgrind

Detects leaks, use-after-free, and invalid memory accesses

**C** AddressSanitizer

Catches out-of-bounds accesses dynamically

**C** Custom Logging

Helps track allocations and custom heap operations

**Java** VisualVM

Visualize heap usage and GC behavior

**Java** JFR

Java Flight Recorder for deep profiling and event recording

**Java** GC Logs

Analyze GC pauses and long-term memory patterns

**Python** tracemalloc

Trace memory blocks allocated by Python internally

**Python** gc module

Introspection and debugging of reference cycles

**Python** Third-party

objgraph and others to analyze object structures

# Summary Table (Conceptual)

## C

(MANUAL)

**ALLOCATION**
malloc/stack choice

**DEALLOCATION**
free by programmer

**SAFETY**
High risk if misused

## Java

(MANAGED)

**ALLOCATION**
new on GC heap

**DEALLOCATION**
GC reclaims unreachable

**SAFETY**
Memory-safe by default

## Python

(MANAGED)

**ALLOCATION**
Objects on managed heap

**DEALLOCATION**
Refcount + GC cycles

**SAFETY**
Memory-safe by default

# Quiz: Identify Stack vs Heap (C)

**Q1** For the code below, identify whether it uses **stack** or **heap**:

```
int x = 5;
int *p = &x;
```

**âœ“ Stack**
Both variables live on the stack. The pointer stores the address of a stack variable.

**âœ— Heap**
Incorrect - no malloc was used here.

**Q2** For the code below, identify whether it uses **stack** or **heap**:

```
double *a = malloc(100 * sizeof(double));
```

**âœ— Stack**
Incorrect - malloc allocates on heap.

**âœ“ Heap**
Pointer 'a' on stack, array on heap. Must call free(a).

# Quiz: Managed Languages

**Q1** What happens when a Java object becomes unreachable from any root?

**âœ“ Correct Answer**
GC may eventually reclaim it when it runs

**âœ— Common Mistake**
It's immediately freed (not true - GC is non-deterministic)

**Q2** Why does Python need both reference counting and a cyclic GC?

**âœ“ Correct Answer**
Cycles keep reference counts non-zero, so cyclic GC is needed

**âœ— Common Mistake**
Reference counting alone is sufficient

**Q3** Can you "force" GC to run in Java or Python? What are the caveats?

**âœ“ Correct Answer**
System.gc() in Java, gc.collect() in Python - but it's a request, not a guarantee

**âœ— Common Mistake**
You can force immediate GC (it's not guaranteed)

# Coding Exercise Ideas in C

## 1 Implement a Simple Dynamic Array

Create a dynamic array with **push**, **pop**, and **reserve** operations using `malloc`/`realloc` and `free`.

```
typedef struct { int *data; size_t size, capacity; } IntVec;
```

## 2 Memory Leak Detection

Write a program that intentionally leaks memory, then fix it using proper cleanup paths.

```
// Intentional leak: missing free(arr)
int *arr = malloc(100 * sizeof(int));
```

## 3 Valgrind/ASan Verification

Use **Valgrind** or **AddressSanitizer** to confirm all allocations are freed and no invalid accesses occur.

# Coding Exercise Ideas in Java and Python

Java

## Implement a Weak Cache

Implement a cache that uses `WeakReference` or `WeakHashMap` and observe how GC affects the stored entries when memory pressure increases.

Python

## Create and Break Reference Cycles

Write code that intentionally creates reference cycles. Use the `gc` module and `tracemalloc` to observe collection triggers and memory usage.

Reflect

## Mental Model Comparison

Reflect on how your mental model of memory differs between C, Java, and Python after completing these experiments. Consider determinism, explicit vs. implicit control, and typical bug patterns.